CreditSense: Loan Risk Assessment Challenge

AI1215 — Project Report

Team Placeholder
Members: Duong Nguyen Canh, Gia Nam Truong

April 2026

1 Project Overview

The CreditSense challenge poses two coupled supervised-learning tasks on the same 35,000-row tabular dataset of personal-loan applicants (55 features per applicant). For every applicant in a 15,000-row test set we predict:

- **Task A — Classification:** `RiskTier` $\in \{0, 1, 2, 3, 4\}$, scored by accuracy.
- **Task B — Regression:** `InterestRate` $\in [4.99, 35.99]$ APR, scored by R^2.

The Kaggle public leaderboard reports a single combined score, Score $= 0.5 \cdot \text{Accuracy} + 0.5 \cdot R^2$. The reported baseline (logistic + linear) is ≈ 0.51.

High-level pipeline. We took an iterative approach across three modeling families:

1. A packaged sklearn pipeline (`train.py` → `src/creditsense/`) that runs an 80/20 stratified holdout, trains baseline linear models on raw features, then trains XGBoost / LightGBM / CatBoost on engineered features and fits a weighted ensemble.
2. A standalone CatBoost-only submission (`train_classic_submission.py`) using Optuna-tuned hyperparameters.
3. A neural-tabular track based on **TabM** — an ensemble of k parallel MLPs sharing input embeddings — searched with Optuna in two iterations (`tune_tabm_v2.py`). The TabM-v2 submission is our **final** approach.

Final result. Our best Kaggle public-leaderboard combined score is **0.88110**, produced by the TabM-v2 model (`submission_tabm_v2.csv`). This beats the logistic/linear baseline by ≈ 0.37 and our best classic-booster submission by ≈ 0.037.

2 Repository and Reproducibility Summary

Source code. https://github.com/duongnc000/MLGroupProject/tree/main

2.1 Layout

```
ML-Assignment/
+-- credit_train.csv, credit_test.csv
+-- src/creditsense/ # packaged sklearn pipeline (config, features,
| modeling, training, __init__)
+-- train.py # entry point for packaged pipeline
+-- train_classic_submission.py # CatBoost-only Optuna-tuned submission
+-- train_tabm_submission.py # TabM v1 submission (LB 0.87417)
+-- train_tabm_kfold_submission.py # TabM v2 submission (LB 0.88110, FINAL)
+-- tune_tabm_v2.py # Optuna search that produced final params
+-- credit_optuna_search.py # Optuna search for sklearn track
+-- artifacts/ # CatBoost tuned params + imputer search CSVs
+-- artifacts_hpo/ # TabM v1 + v2 tuned params
```

```
+-- outputs/ # data_profile.json, experiment_results.csv,
| run_summary.json
+-- submission.csv # TabM v1 (LB 0.87417)
+-- submission_classic.csv # CatBoost (LB 0.84443)
+-- submission_tabm_v2.csv # TabM v2 (LB 0.88110, FINAL)
+-- notebooks/eda_starter.ipynb # exploratory notebook
+-- README.md, requirements.txt
```

2.2 How to reproduce the final submission

1. Create a Python 3.10+ environment.
2. Install all dependencies: `pip install -r requirements.txt` (covers the sklearn / XGBoost / LightGBM / CatBoost stack as well as PyTorch, Optuna, TabM, and `rtdl_num_embeddings`).
3. Run `python train_tabm_kfold_submission.py`. The script loads tuned hyperparameters from `artifacts_hpo/tabm_v2_best_params_*.json`, trains TabM on the full training set for both tasks, and writes `submission_tabm_v2.csv`.
4. (Optional) re-run the search: `python tune_tabm_v2.py` regenerates the JSONs in `artifacts_hpo/`.

2.3 Caveats and reproducibility risks

- Random seeds differ across tracks: the packaged sklearn pipeline uses `RANDOM_STATE=1215` (defined in `src/creditsense/config.py`); the CatBoost and TabM scripts use `SEED=42`. Within a track, results are deterministic; across tracks, exact values are not directly comparable.
- Validation regimes differ: the packaged pipeline uses an 80/20 stratified holdout; the Optuna searches use 3-fold CV (StratifiedKFold for classification, KFold for regression). Both protocols are reported below.
- GPU is preferred for TabM (`tune_tabm_v2.py` auto-selects `cuda` if available); CPU works but is much slower.

3 Data Exploration and Preprocessing

3.1 Dataset shape

The training file `credit_train.csv` contains 35,000 rows and 55 features plus two target columns. The test file `credit_test.csv` has 15,000 rows with the same features. Features fall into five groups (per the assignment brief): Demographics (8), Employment & Income (10), Assets & Liabilities (9), Credit History (18), Loan Request (10).

3.2 Missingness analysis

The packaged pipeline writes `outputs/data_profile.json` via `build_data_profile` in `src/creditsense/featu`
The profile contains per-column missingness counts and basic numeric/categorical typing.

Meaningful-missing columns. `src/creditsense/config.py` explicitly lists five columns where missingness encodes a category rather than measurement error:

```
MEANINGFUL_MISSING_COLUMNS = [
    "PropertyValue", # missing for renters
    "MortgageOutstandingBalance", # missing for non-homeowners
    "StudentLoanOutstandingBalance", # missing for older applicants
    "CollateralValue", # missing for unsecured loans
    "SecondaryMonthlyIncome", # missing for single-earner households
]
```

These five fields drive the missing-flag feature engineering (Section 4). Figure 2 shows the per-column missingness across the training set, with the meaningful-missing columns highlighted in red.

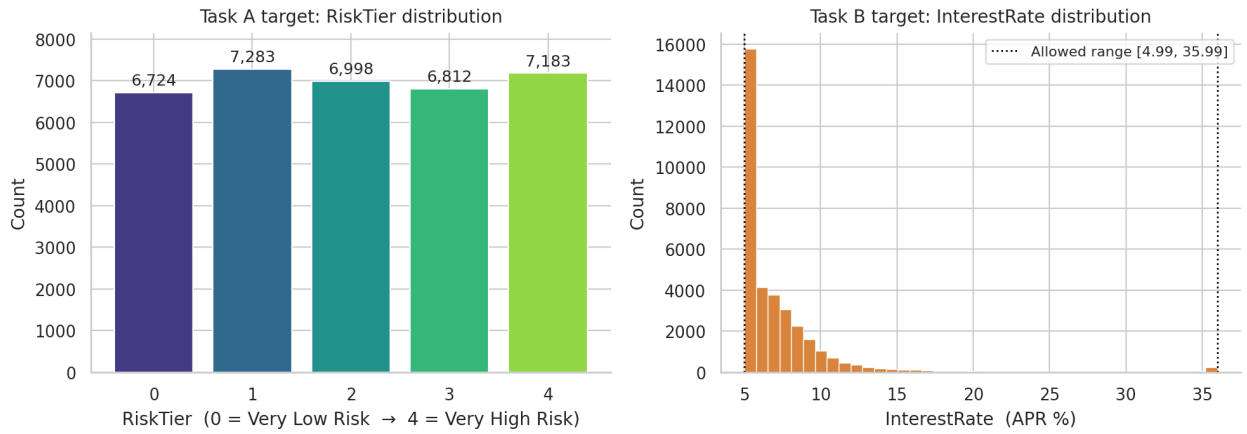

Figure 1: Target distributions in the training set. Left: `RiskTier` classes are roughly balanced, so plain accuracy is a fair metric. Right: `InterestRate` is right-skewed but stays inside the allowed $[4.99, 35.99]$ range, motivating both the export-time clipping and the log-transform option used by TabM.

3.3 Validation strategy

- **Packaged sklearn pipeline:** a single 80/20 stratified holdout (`TEST_SIZE=0.2`, `RANDOM_STATE=1215`, stratified on `RiskTier`). Both tasks share the same split, so the combined local score $0.5 \cdot \text{Acc} + 0.5 \cdot R^2$ is internally comparable.
- **Optuna searches** (`credit_optuna_search.py`, `tune_tabm_v2.py`): 3-fold cross-validation. Classification uses `StratifiedKFold`; regression uses `KFold`. Both with `shuffle=True` and `SEED=42`.

3.4 Imputation and encoding

Three model families use three distinct preprocessing strategies. This is intentional: gradient-boosting trees and embedding-based neural nets benefit from different input formats.

Family	Numeric NA	Categorical NA	Encoding
Linear baselines (sklearn pipeline)	median impute	most-frequent impute	one-hot (`handle_unknown=ignore`)
Gradient boosters (XGB/LGB/CatBoost)	median impute	most-frequent / native CatBoost handling	one-hot or native categorical
TabM neural ensemble	Optuna-chosen (mean / median / KNN / iterative)	Optuna-chosen (most-frequent / dedicated "special" bucket)	`OrdinalEncoder` shifted by +1 so index 0 reserves the missing/unknown bucket

3.5 Exploratory notebook

`notebooks/eda_starter.ipynb` contains the initial data-exploration notebook (loaded into Jupyter or Colab). It is not executed by the pipelines; it is reference material.

4 Feature Engineering

Each modeling track has its own feature-engineering implementation; column names diverge across tracks but the underlying intuitions are similar. We do not attempt to merge them — each track was tuned end-to-end against its own preprocessing.

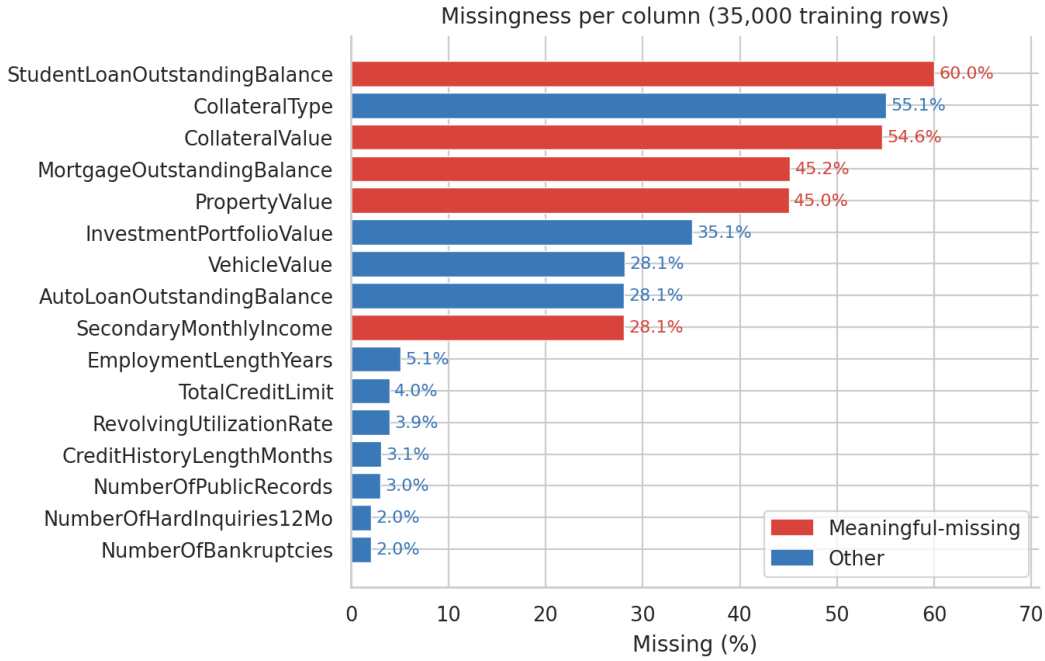

Figure 2: Per-column missingness over the 35,000 training rows. The five columns highlighted in red are the ones whose NaN value encodes a real category (renter has no `PropertyValue`, etc.); these are preserved through explicit `*_missing_flag` indicators rather than being imputed away.

4.1 Packaged sklearn pipeline (`src/creditsense/features.py`)

The function `add_engineered_features` appends, conditional on each input column existing:

- **Missing-flag features** for every column in `MEANINGFUL_MISSING_COLUMNS` (suffix `_missing_flag`).
- **Aggregate balances and ratios:** `total_outstanding_debt` (sum of mortgage/auto/student loan), `debt_to_assets`, `unused_credit_ratio`, `income_per_dependent`, `loan_to_assets`, `requested_payment_credit_age_ratio`.
- Helper `safe_divide` guards against division by zero.

4.2 TabM track (`tune_tabm_v2.py`)

The TabM script defines four feature groups, with a top-level switch `features` chosen by Optuna from {`None`, `basic_credit`, `basic_credit_plus_logs`}:

- `_add_basic_credit_features`: `LiquidAssets` (Savings + Checking), `TotalDebtApprox`, `RequestedToAnnualI PaymentToIncomeRatio_FE`, `TotalDebtToIncome`, `CollateralCoverage`.
- `_add_credit_behavior_features`: `TotalLatePayments` (30+60+90 days), `WeightedLatePayments` ($1\cdot30$d + $2\cdot60$d + $3\cdot90$d), `AnyDerogatoryFlag` (bankruptcies + public records + collections + charge-offs > 0).
- `_add_time_features`: `CreditHistoryYears` = `CreditHistoryLengthMonths`/12.
- `_add_log_money_features`: `log1p`-transformed money columns (income, balances, requested loan, collateral, monthly payment).

The Optuna-selected feature bundles per task are reported in Section 5.3.

5 Model Development

5.1 Why TabM?

The two strongest model families on tabular data are gradient-boosted decision trees and, more recently, parameter-efficient neural ensembles. We exhausted the boosting side first (XGBoost, LightGBM, CatBoost, all Optuna-tuned) and saw the public-LB score plateau around 0.844. To

push further we adopted **TabM** [1], a recent neural architecture designed specifically for tabular data, together with the `rtdl_num_embeddings` library [2] that supplies its numeric input layer.

What TabM is. TabM [1] is a *parameter-efficient ensemble of k multilayer perceptrons*. The k MLPs share their input embeddings and feature backbone but each has its own prediction head, so a single forward pass produces k slightly different outputs. At inference these are averaged, giving an in-model bagging effect for a fraction of the cost of training k separate networks. The architecture is one of the few neural designs that has been shown to compete with — and on some benchmarks beat — gradient-boosted trees on tabular problems.

What `rtdl_num_embeddings` is. A small support library [2] that replaces naive scalar inputs with *piecewise-linear numeric embeddings*: each numeric feature is binned into `n_bins` pieces and a learned vector is associated with each bin (interpolated linearly within each piece). This lets the MLP capture non-linear thresholds (for example, "DTI above 0.4 is bad") from the start instead of having to learn them from scratch through deep stacks of linear layers. TabM expects this kind of modern numeric encoding rather than raw scalar inputs, which is why the library is a hard dependency of our final pipeline.

5.2 Models tried per task

Family	Classification (Task A)	Regression (Task B)
Linear baseline	Logistic Regression	Elastic Net
Gradient boosters	XGBoost, LightGBM, CatBoost	XGBoost, LightGBM, CatBoost
Within-track ensemble	Tuned weighted blend (XGB + LGB + LR)	Tuned weighted blend (CatBoost + XGB + LGB)
Neural tabular	TabM v1, TabM v2	TabM v1, TabM v2

5.3 Hyperparameter tuning with Optuna

Both Optuna searches use **TPE sampler** (multivariate, with grouping) and a **Median pruner** (`n_startup_trials=8`, `n_warmup_steps=1`) so that unpromising trials are aborted mid-CV.

CatBoost (`credit_optuna_search.py`). Search space includes `iterations`, `depth`, `learning_rate`, `l2_leaf_reg`, `random_strength`, `bagging_temperature`, `border_count`, plus imputer choices. Best parameters saved to `artifacts/best_params_{classification,regression}.json`:

```
classification: iterations=2812, depth=6, lr=0.0366, l2_leaf_reg=5.53,
              random_strength=2.37, bagging_temperature=0.41, border_count=220
regression: iterations=3800, depth=6, lr=0.0167, l2_leaf_reg=1.40,
              random_strength=1.11, bagging_temperature=0.64, border_count=183
```

TabM v2 (`tune_tabm_v2.py`). 60 trials per task, 3-fold CV. Search space spans preprocessing (numeric and categorical imputers, scaling, feature bundle), numeric embedding (`piecewise-linear`, `d_embedding`, `n_bins`), architecture type (`tabm` vs `tabm-mini`), and training (`k` heads, `n_blocks`, `d_block`, dropout, lr, weight decay, batch size, epochs, patience, cosine schedule with warmup, gradient clipping, label smoothing). Best parameters saved to `artifacts_hpo/tabm_v2_best_params_*.json`:

5.4 Overfitting controls

- **Cross-validation in tuning:** 3-fold CV scores drive Optuna; the median pruner kills trials that fall below the running median after the first fold.

Hyperparameter	RiskTier (cls)	InterestRate (reg)
`features`	`None`	`basic_credit`
`numerical_imputer`	`mean`	`median`
`categorical_imputer`	`most_frequent`	`special` (own bucket)
`num_embedding`	`piecewise-linear`, d=8, n_{bins}=16	`piecewise-linear`, d=16, n_{bins}=32
`arch_type`	`tabm`	`tabm-mini`
`k` (heads)	48	48
`n_blocks`, `d_block`	5, 320	5, 320
`dropout`	0.045	0.104
`lr`, `weight_decay`	1.5e-3, 3.4e-4	3.4e-4, 1.9e-3
`batch_size`, `n_epochs`, `patience`	256, 53, 15	256, 122, 23
LR schedule	cosine, no warmup	no cosine, 5-epoch warmup
`label_smoothing`	0.044	—
CV best (_best_value)	**0.8895** (Acc)	**0.8494** (R^2)

- **TabM internal ensemble:** the k=48 parallel heads act as in-model bagging — inference averages across heads, smoothing single-seed noise.
- **Regularization:** dropout, weight decay, label smoothing, gradient clipping (TabM v2); `l2_leaf_reg`, `bagging_temperature`, `random_strength` (CatBoost).
- **Early stopping:** TabM uses Optuna-chosen `patience`; CatBoost uses a capped `iterations`.
- **Train/val separation:** the packaged pipeline never trains on its holdout; the Optuna searches never train on a CV fold's validation portion.

6 Results

6.1 Holdout validation (sklearn pipeline)

From `outputs/experiment_results.csv` (80/20 stratified holdout, seed 1215):

Configuration	Task A Acc	Task B R^2	Combined
Logistic / Elastic-Net (raw features, baseline)	0.634	0.618	0.626
XGBoost + engineered features	0.792	0.840	0.816
LightGBM + engineered features	0.791	0.839	0.815
CatBoost + engineered features (regressor)	—	0.846	—
Tuned weighted ensemble (engineered)	**0.798**	**0.847**	**0.822**

The classic-track summary is captured in `outputs/run_summary.json`: best classification model = tuned weighted ensemble (val Acc 0.798); best regression model = tuned weighted ensemble (val R^2 0.847); combined local score 0.822.

6.2 CatBoost cross-validation (Optuna)

From `artifacts/imputer_search_{classification,regression}.csv`, top row of each (best imputer + feature combo found):

Task	Best CV score	Imputer (numeric / categorical)
Classification (Acc)	0.816	mean / most_frequent
Regression (R^2)	0.837	mean / special

6.3 TabM cross-validation (Optuna)

From the `_best_value` field in `artifacts_hpo/tabm_v2_best_params_*.json`:

Task	TabM v2 CV score	TabM v1 (no CV recorded)
Classification (Acc)	0.8895	—
Regression (R^2)	0.8494	—
Combined CV mean	0.8695	—

6.4 Kaggle public leaderboard (final scoring)

Submission file	Source script	Public LB
Baseline (template, reference only)	—	≈ 0.51
`submission.csv` (sklearn ensemble baseline)	`train.py` family	0.82717
`submission_classic.csv`	`train_classic_submission.py`	0.84443
`submission.csv` (TabM v1)	`train_tabm_submission.py`	0.87417
`submission_tabm_v2.csv` (final)	`train_tabm_kfold_submission.py`	**0.88110**

Note on attribution. Public-LB scores above are user-reported from the Kaggle competition page; they are not automatically reproduced by the repository. The submission CSV files themselves are committed (the predictions Kaggle scored), but the leaderboard positions are external metadata.

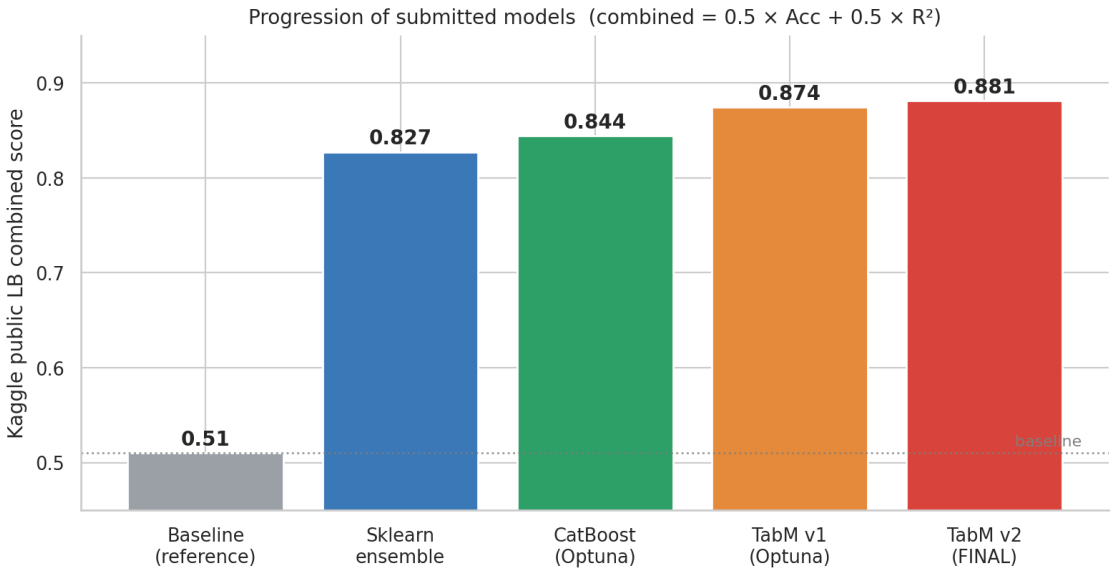

Figure 3: Public-leaderboard combined score by submitted model. Each step represents a meaningfully different modeling family (sklearn ensemble → tuned CatBoost → TabM v1 → TabM v2). The TabM v2 submission improves on the assignment baseline by ≈ 0.37, and on our best classic-booster submission by ≈ 0.037.

6.5 Did the same features matter for both tasks?

No. Optuna selected different feature bundles for the two TabM models: the classifier preferred raw features (`features=None`) with cosine LR + label smoothing; the regressor preferred the `basic_credit` bundle with a dedicated missing bucket and warmup-only schedule. The CatBoost track also picked different imputers per task (`most_frequent` vs `special`). The final per-task feature/preprocessing choices are not shared across tasks; only the model family is shared.

7 Strengths, Weaknesses, and Next Improvements

7.1 Strengths

- **Multiple validation regimes.** The packaged pipeline holds out a fixed 20% slice; Optuna runs 3-fold CV. Both regimes agree on the model ranking.
- **Tuned hyperparameters preserved.** Every modeling track that was used to produce a committed submission has its parameters serialized to JSON in `artifacts/` or `artifacts_hpo/`, so submissions can be regenerated from source + data + JSON without re-running the search.
- **Meaningful-missing handling.** `MEANINGFUL_MISSING_COLUMNS` is a single source of truth for the columns where NaN encodes a real category; missing-flags and the TabM "special" categorical bucket both consume it.
- **Clear progression.** Baseline (≈ 0.626 combined) $\rightarrow$ engineered features + boosters (≈ 0.822) $\rightarrow$ Optuna-tuned CatBoost (LB 0.844) $\rightarrow$ Optuna-tuned TabM (LB 0.881). Each step adds verifiable evidence.

7.2 Weaknesses

- **Duplicated feature-engineering logic.** `src/creditsense/features.py` and `tune_tabm_v2.py` each define their own ratio/aggregate features with overlapping but non-identical column names (`total_outstanding_debt` vs `TotalDebtApprox`, etc.). Changing one does not update the other.
- **Inconsistent random seeds across tracks** (1215 vs 42). Within a track, results are deterministic; cross-track local comparisons are approximate.
- **Blend track removed.** A weighted TabM + CatBoost blend was attempted (originally in `train_ensemble_submission.py`) but its public-LB score was lower than pure TabM, so the script was excluded from the commit. The negative result is documented here but no diagnostic data was saved.

7.3 Next improvements

- Consolidate feature engineering into `src/creditsense/features.py` and have all training scripts import the same `add_engineered_features` (with an opt-in flag for log transforms).
- Try stacking instead of blending: train a lightweight meta-learner on TabM and CatBoost out-of-fold predictions, rather than fixed convex weights.
- Run TabM with a wider `k` sweep (current best is k=48); if compute allows, k=64–96 with stronger weight-decay may help.

References

[1] Gorishniy, Y., Kotelnikov, A., Babenko, A. (2024). *TabM: Advancing Tabular Deep Learning with Parameter-Efficient Ensembling*. arXiv preprint arXiv:2410.24210. https://arxiv.org/abs/2410.24210.

[2] Gorishniy, Y., Rubachev, I., Babenko, A. (2022). *On Embeddings for Numerical Features in Tabular Deep Learning*. Advances in Neural Information Processing Systems (NeurIPS) 35. arXiv:2203.05556. https://arxiv.org/abs/2203.05556.